Embedded Systems

PROG4-Lab
final assignment

Peter Bijl 12 April 2023

Version 1.0

Document history

Date	Version	Who	Changes
12-04-2023	1.0	Peter Bijl	First version.

HAN University of Applied Sciences

Embedded Systems

CONTENTS

Preface 4
1. Preparing and carrying out the final assignment 5
   1.1 Preparing additional research on C++ and GUI widgets 5
   1.2 General requirements for completion 5
2. Final assignment 6
   2.1 Operating software without hardware with GUI 6
   2.2 Final assignment 6
3. Requirements for the product document 8
   3.1 Content of the product report 8
   3.2 Product report: layout and style requirements 10
   3.3 Reviewing: ask others to read your product report 10
   3.4 Schedule 11

PREFACE

This document contains the texts that accompany the final assignment for PROG4. In the final assignment, detailed use should be made of the programming knowledge and skills in C++ and the use of Qt that were acquired in PROG4. The focus is on building a complete GUI in a C++ program for controlling a device, similar to the one made in C in PROG2. In addition to the program, you need to write a product report to complete the course.

Arnhem, April 2023
Peter Bijl

1. PREPARING AND CARRYING OUT THE FINAL ASSIGNMENT

This chapter discusses the requirements for the approach to and development of the final assignment. Links are expressly drawn with professional practice. Think of your lecturer as your client. In the practicals, you should show your (working) interim results on a regular basis, so that your client (the lecturer) can give you feedback.

1.1 Preparing additional research on C++ and GUI widgets

The preparation mainly consists of studying C++ and Qt widget code samples (available on OnderwiisOnline).

There are many widgets with all kinds of settings for building a GUI in C++. It is essential to study existing GUI code samples and to re-use them (in part) for your own GUI. This is what is done in professional practice. See the example code for an overview of widgets that might be suitable. Their use in your own code should be derived from the sample code.

1.2 General requirements for completion

The following requirements apply to the completion of the final assignment:

- **Works well**
  - program can be started up and does not crash while in use
  - has demonstrably been tested (run through at least all states visible in log)
  - program meets the specifications

- **User-friendliness**
  - informative, business-like construction of GUI and appropriate use of color for widgets
  - tested for incorrect input with recovery option for wrong input
  - GUI implemented in the English language

- **Simplicity, reliability, functionality**
  - no-frills approach
  - does not include any non-specified functionality

- **Clear classification of software in directories**
  - use of a single directory for the final assignment, with a suitable name

- **Clear and logical program layout**
  - program code should have a logical layout
  - use of code formatter in Qt Creator

- **Readable, informative C++ code**

2. FINAL ASSIGNMENT

This chapter provides an overview of the steps needed to convert the code made in PROG2 to C++ and to provide it with a GUI in the Qt Creator development environment.

2.1 Operating software without hardware with GUI

To build the GUI, we need to start from the results of research into existing widgets. You can find a lot of examples on the internet.

You can also select potentially suitable widgets from the **Qt Examples**. Only select widgets that have been made with C++ and not with QML.

See also, for example: http://doc.qt.io/qt-5/examples-widgets.html

Copy the suitable files that you require into your own Qt project. Do your own research into other potential sources of widgets. Talk to your lecturer if you are unsure about whether the code samples are appropriate. It is risky to choose samples that are too complex.

2.2 Final assignment

Development steps and requirements:

- Build the GUI in English.
- Do research into the required widgets.
- Make a sketch of the GUI with appropriate widgets. Choose a suitable layout structure for this (horizontal, vertical, grid and hierarchical combinations). Indicate the structure clearly.
- First, implement the most important widgets. Work step by step. The code should work after every step.

- Study the working of the CVM-GUI example. Pay detailed attention to the implementation of the state machine in the State Machine class. Implement your own state machine, which should be able to respond correctly to GUI events and show messages in a display.

!	For debugging purposes, the console can be used at any time to display messages with `cout << … .`

In practice, use is often made of a log to display and/or save the behaviour of the application in a standard way. **Logging is not intended for the user, but for the device's administrator or service engineer.** There are different types of messages, such as: debug, info, warning, critical and fatal.

- Do internet research into the different logging levels (debug, info, warning, ... etc.). In this way, you can decide how to use them in your own application.
- Use the class QMessageLogger to facilitate logging. Choose all kinds of places in the application to generate informative log messages of the correct type. At a minimum, implement debug and info logging.
- In addition to the display, you also need to use an extra display to display the debug log messages.
- All logging should be written to a standard log file.
- Make it possible to turn off the log display with a button. This could be an action that is carried out by the device's administrator or service engineer.

Optional (in consultation with lecturer, if sufficient time remains): because state machines are often used, there is also a state machine framework in Qt. A thorough knowledge of C++ is required for this. The main focus is the class QStateMachine. https://doc.qt.io/qt-6/qtstatemachine-index.html

- Save the "old" application and work in a duplicate version for the next assignment.
- Replace the state machine with the Qt state machine framework. Do internet research on code samples for this.

!	Build the GUI in small steps and always save a copy of the working code. Avoid using extensive and complex additions, as your code may no longer work well. Always make back-ups of your programs and documents.

3. REQUIREMENTS FOR THE PRODUCT DOCUMENT

The requirements for the product document are the same as those for PROG2. This means that a number of chapters can be copied, to a great extent. You will need to make adjustments and additions, of course, based on the new requirements and the use of a GUI.

The product report should provide information for the client and developers. This will allow them to adapt the program to new requirements, add to it, improve it and/or get rid of errors (bugs) at a later stage. In practice, a lot of time is spent on the further development of programs, mostly by people other than the ones who made the first version! This means that the quality of the product documents should be such that they are comprehensive and unambiguous. If they are not, the cost of developing subsequent versions of the program can become needlessly high.

3.1 Content of the product report

The product report should contain the following numbered chapters (if a chapter becomes too long, you can split it into numbered sections with informative section headings):

Title page
Summary
Preface
Table of contents

1 Introduction
2 Definition phase
3 Design phase
4 Realization phase
5 Test plan
6 End result

Appendix 1 Program listings
Appendix 2 User manual

- The **title page** should contain the project title (name of the product developed) and sub-title (name of the relevant INFT2 study unit), author with student number, class, name of supervising lecturer, the date and possibly an appropriate illustration.

- The **summary** should cover a maximum of three-quarters of a page and **it should be able to be read completely separately from the report** (i.e. you should assume that the reader is unfamiliar with the project). This summary should cover the entire project. You should give a short description of what

the project is about: definition of the problem that was solved for the client, the client, the most important requirements and which results have been achieved/not achieved, and finally, potential recommendations for improving and/or extending the program. You should also give a short indication of the conclusion of the tests that have been performed. **The summary may not include any references to the rest of the document.**

- The **preface** is intended for **personal comments**. For example, you could include something about the experience you acquired during the project (what have you learned, content-wise?), the problems that arose, what caused delays and such. Finally, you should indicate what you would do better next time. You may write the preface in first person (I style).

- The **table of contents** lists all the numbered chapters and sections of the report with the page number at the end of the line.

1. The **introduction** gives a succinct description of what the report is about and how it is structured. By reading the introduction, readers should be able to familiarize themselves with the content of the report and determine whether the content is relevant enough to continue reading. The reader may decide to read only certain chapters or chapter sections. This means that you should discuss, in a concise but informative way, which topic and problems will be addressed and solved and what will be addressed in the consecutive chapters.

2. The **definition phase** describes the project's background, the relationship with the previous PROG2 project, the main function and the requirements and wishes relating to the product under development. The definition phase results in a functional specification and a technical specification. The functional specification is about the external aspects of the system, the 'what' (describe the behavior, what the system does for its users and/or other systems). Number the specifications. The technical specifications cover, for example, the choice of the ISO-C++ programming language and the Qt widgets library.

3. The **design phase** describes the internal aspects of the system (the processing side, or the 'how'). This gives a global indication of how the functional specification is converted into a technical solution with a GUI. Describe the user interface (choice of keys and display) with the aid of screen dumps. Briefly explain what should happen after certain rescue actions.

   Furthermore, the limits are defined (what it can/can't do) and the acceptance test is described (to show the client whether the system meets the specifications). For this purpose, add a sketch of the detailed and adjusted state machine with all names for the events in the transitions. **The names of events should be consistent with the code.**

4. The **realization phase** describes the detail design (pseudocode or flowchart and program code) and provides a user manual. You can use appendices for

this. In the realization phase, detailed test cases are also generated to test the product in depth. The test plan is discussed in Chapter 5.

5. The **test plan** shows the test cases for using the GUI (user level) and the acceptance test with results. Display the tests in the form of a table.

6. The **end result** covers
   - Draw the stratified software architecture

- **Appendix 1 Program listings** contains the listings that belong with the application, i.e. also the header file(s). Order: MainWindow class, StateMachine class, one or more combinations of remaining header files with corresponding implementation files. The files with external code for the widgets should **not** be added. Do include a reference to these.

- **Appendix 2 User manual** contains the user manual that accompanies the selected user interface. This should be in English. Ensure that the user has an account of all the states in the state machine that provide input and output. Be aware that the content should not be described in a technical way; this is a user manual. Pay attention to starting up, using and switching off the application.

  For administrators or service engineers, explain how to use the log.

3.2 Product report: layout and style requirements

Layout and style requirements:

- Number the pages, chapters and sections.
- Start every chapter on a new page.
- Give the sections informative headings so it is easy to find a particular topic in the table of contents.
- Use a business-like style, i.e. do not use first person singular or plural (except in the preface).
- Pay attention to correct spelling and syntax. Always use the spelling and grammar check in the word processing program.
- Use lists rather than long sentences, allowing the reader to search easily for different subjects.
- Put code in a monospaced font such as Courier New, which makes it much easier to read. Always use a logical, correct, readable layout.

3.3 Reviewing: ask others to read your product report

We know from practice that writing is difficult. This is also the case for experienced writers. Having someone read through (review) your work provides rapid insight into the quality of the document.

3.4 Schedule

You are expected to start working on a draft of the product report in week 4. It is essential to get feedback on this from the lecturer. Bear in mind that the lecturer will have limited time.

This educational publication does not contain any copyrighted material.